



University of Science - VNUHCM

Faculty of Information Technology

Software Testing

Software Testing Project Report

HW07 - API Testing

Authors:

Võ Lê Việt Tú (22127435)
vlvtu22@clc.fitus.edu.vn

Supervisors:

Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

August 25, 2025

Table of Contents

1	Group Information	1
2	Overview	1
3	Test execution	2
3.1	Set up environment	2
3.1.1	Steps to Create an Environment	2
3.2	API 01 - POST /users/register	3
3.2.1	Overview	3
3.2.2	How to Test with Postman	4
3.2.3	Example Test Cases	5
3.2.4	Expected Responses	5
3.2.5	Additional Notes	5
3.3	API 02 - GET /products	5
3.3.1	Overview	5
3.3.2	How to Test with Postman	6
3.3.3	Example Test Cases	6
3.3.4	Expected Responses	6
3.3.5	Additional Notes	6
3.4	API 03 - GET /products/{productId}	7
3.4.1	Overview	7
3.4.2	How to Test with Postman	8
3.4.3	Example Test Cases	8
3.4.4	Expected Responses	8
3.4.5	Additional Notes	9
4	AI supports	9
4.1	Test Case Generation	9
4.1.1	Example Prompt for Test Case Generation	9
4.2	Test Collection Generation	9
4.2.1	Example Prompt for Test Collection Generation	9
4.3	Summarize Defects and Actual Results	10
4.3.1	Example Prompt for Summarizing a Bug	10
4.3.2	Generated Output Example	10
5	Test results	10
5.1	API 01 - POST /users/register	10
5.1.1	Evidence	12
5.2	API 02 - GET /products	16
5.2.1	Evidence	17
5.3	API 03 - GET /products/{productId}	21
5.3.1	Evidence	22

6	API Testing CI Integration	25
6.1	Overview	25
6.2	Implementation	25
6.2.1	Technology Stack	25
6.2.2	Workflow Architecture	26
6.2.3	Test Coverage	26
6.3	Technical Configuration	26
6.3.1	Newman Execution	26
6.3.2	Environment Setup	26
6.3.3	Reporting	26
6.4	Results	26
6.4.1	Evidence	27
7	Self-assessment	29

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Features	Status
Cao Uyên Nhi	22127310	- POST /favorites - POST /brands - POST /payment/check	Done
Lưu Thanh Thuý	22127410	- POST /users/login - POST /users/register (admin only) - PUT /users/{id} (admin only)	Done
Nguyễn Phước Minh Trí	22127424	- POST /products - PUT /products/{productId} - GET /products/{productId}/related	Done
Võ Lê Việt Tú	22127435	- POST /users/register - GET /products - GET /products/{productId}	Done
Trần Thị Cát Tường	22127444	- POST /messages - POST /categories - PUT /categories/{categoryId}	Done

2 Overview

In **Sprint 5** of the Tool Shop project, we validated core APIs using **Postman** and automated the checks in **GitHub Actions** (CI) via **Newman**. The focus was on functional correctness, schema compliance, error handling, and regression safety. APIs exercised:

- **Register User** – POST /users/register
- **Get All Products** – GET /products
- **Get Specific Product** – GET /products/{productId}

Key activities:

- Organized requests into **Postman Collections** with **environments** ({{baseUrl}}).
- Used **pre-request** and **test** scripts for assertions and data-driven runs (Collection Runner).
- Integrated with **GitHub Actions**: on each push/PR, Newman runs the collection, publishes console logs and HTML reports as CI artifacts, and surfaces pass/fail in the PR.

Notable findings (Sprint 5):

- **Incorrect status codes** for valid inputs (e.g., 422 on Register, 404 on valid product IDs).

- **Schema mismatches** (e.g., `price` returned as string).
- **Missing fields** in product list responses.
- **Intermittent server errors** under higher data volumes.

Outcome:

- Clear visibility into endpoint behavior and regressions.
- CI-backed feedback loop ensures issues are caught early and tracked between commits.

3 Test execution

3.1 Set up environment

To make API testing more flexible and maintainable, we used **Postman Environments**. Environments allow us to store and reuse variables across multiple requests, reducing duplication and making the collection portable.

3.1.1 Steps to Create an Environment

1. Open Postman and click on the **Environments** icon (gear symbol in the top right).
2. Select **Add Environment**.
3. Define environment variables such as:
 - `baseUrl` → `http://localhost:8091`
4. Save the environment and select it from the environment dropdown.
5. Use variables in requests with the `{{variableName}}` format.

Example:

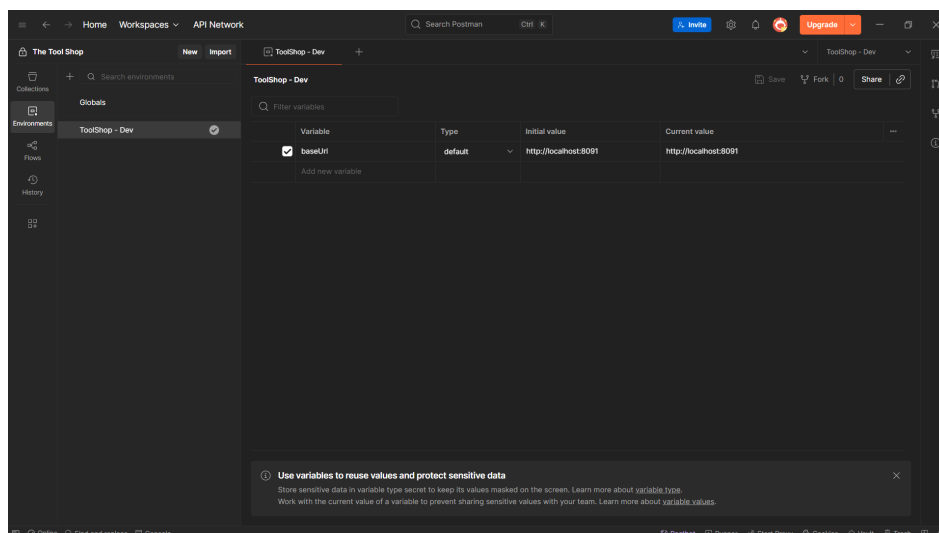


Figure 1: Postman Environment Setup with Variables

3.2 API 01 - POST /users/register

3.2.1 Overview

This API endpoint allows new users to register by submitting their personal information. The endpoint expects a JSON payload with required fields and returns a unique user ID upon successful registration.

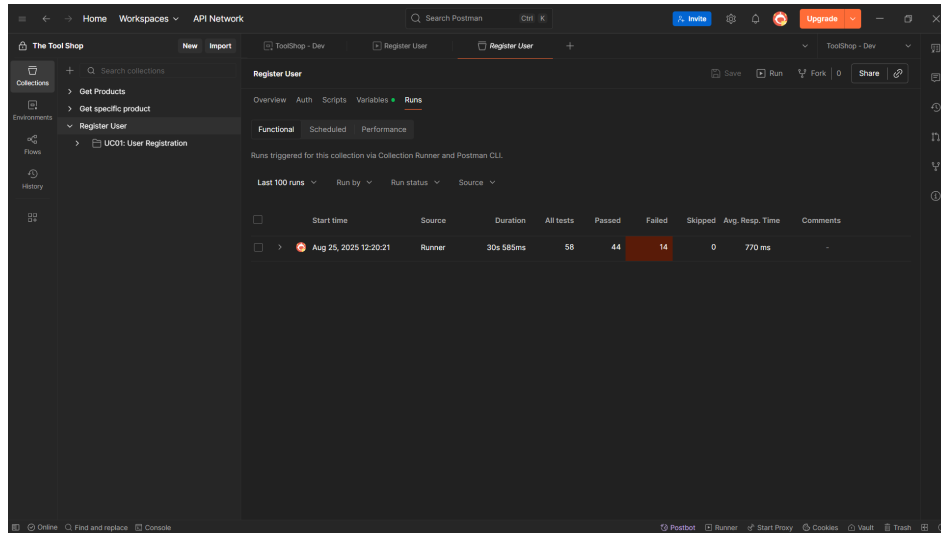


Figure 2: Users register collection in Postman

3.2.2 How to Test with Postman

1. Set Up Environment Variables

- Define `baseUrl` in your Postman environment (e.g., `http://localhost:3000` or your API host).
- The test uses a dynamic email to avoid duplicate registration errors.

2. Request Details

- **Method:** POST
- **URL:** `{{baseUrl}}/users/register`
- **Headers:**
 - Content-Type: `application/json`
- **Body (raw, JSON):**

```
{
  "first_name": "John",
  "last_name": "Doe",
  "phone": "0987654321",
  "dob": "1970-01-01",
  "password": "aSecureAndRandomPwd_2024!",
  "email": "{{randomUserEmail}}",
  "address": "Street 1",
  "city": "City",
  "state": "State",
  "country": "Country",
  "postal_code": "1234AA"
}
```

- **Pre-request Script:**
Generates a random email for each run:

```
const randomEmail = `john.doe.${Date.now()}@example.com`;
pm.globals.set("randomUserEmail", randomEmail);
```

3. Test Scripts

- **Success Case:**
Checks for status code 201 and verifies the response contains a unique user ID and correct data.
- **Negative Cases:**
 - Duplicate email returns 400/409/422 and an error message.
 - Missing required fields (e.g., email) returns 400/422 and a relevant error.
 - Invalid email formats are rejected with 400/422 and a validation error.

3.2.3 Example Test Cases

- **TC101:** Successful registration with all valid fields.
- **TC102:** Attempt to register with a duplicate email.
- **TC103:** Registration missing required field (email).
- **TC104:** Registration with invalid email format (missing '@').
- **TC105:** Registration with invalid email format (missing domain).

3.2.4 Expected Responses

- **201 Created:**
Registration successful. Response includes user data and a unique id.
- **400/409/422 Error:**
Registration failed due to validation errors, duplicate email, or missing fields. Response contains error details.

3.2.5 Additional Notes

- Always use a unique email for each registration test to avoid false negatives.
- The API should validate all required fields and email format.
- Error messages should be clear and indicate the specific issue (e.g., “email already exists”, “invalid email format”).
- All tests are automated in Postman using scripts for both pre-request and test validation.

3.3 API 02 - GET /products

3.3.1 Overview

This endpoint retrieves a paginated list of all available products. It supports filtering by brand, category, price range, and rental status, as well as sorting by name or price.

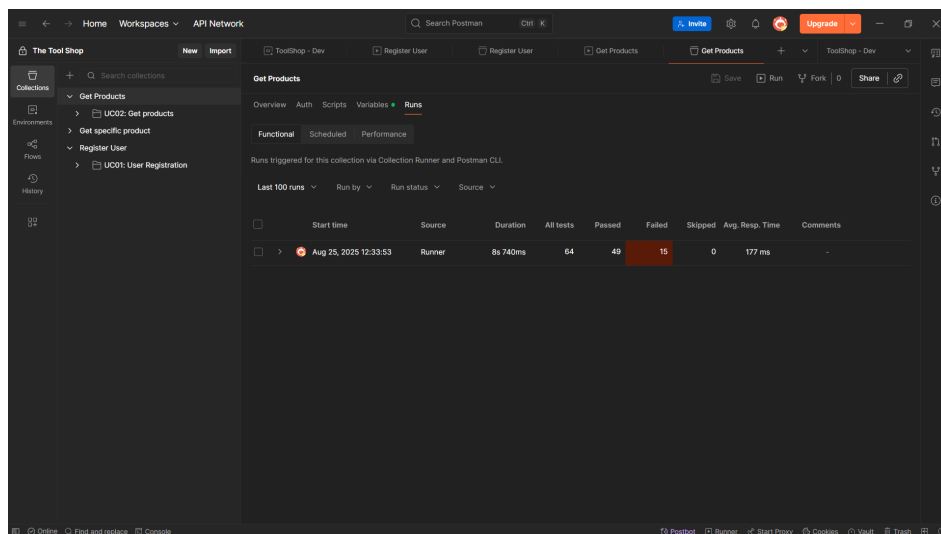


Figure 3: Get products collection in Postman

3.3.2 How to Test with Postman

1. Request Details

- **Method:** GET
- **URL:** `{{baseUrl}}/products`
- **Headers:**
 - **Accept:** `application/json`

2. Query Parameters for Filtering and Sorting

- **by_brand={id}:** Filters products by brand ID.
- **by_category={id}:** Filters products by category ID.
- **between=price,{min},{max}:** Filters products within a price range.
- **is_rental={1|0}:** Filters for rental or purchase-only tools.
- **sort={name|price},{asc|desc}:** Sorts products by name or price.

3. Test Scripts

- **Happy Path (TC201):** Checks for a 200 status and validates the pagination structure (`current_page`, `data`, `total`).
- **Filtering (TC202-TC206):** Verifies that the returned products match the filter criteria (e.g., all products have `brand.id` equal to the requested ID).
- **Sorting (TC207-TC210):** Confirms that the product list is correctly sorted in ascending or descending order based on the specified field.

3.3.3 Example Test Cases

- **TC201:** Get all products without any filters.
- **TC202:** Filter by Brand ID 1.
- **TC203:** Filter by Category ID 2.
- **TC204:** Filter by price between 10 and 50.
- **TC207:** Sort by name in ascending order.
- **TC208:** Sort by name in descending order.

3.3.4 Expected Responses

- **200 OK:** Request is successful. The response body contains a paginated list of products.
- **4xx/5xx Error:** Indicates an issue with the request or a server-side problem. The response should contain a descriptive error message.

3.3.5 Additional Notes

- The tests include helper functions to safely parse JSON and extract the data array, ensuring that tests do not fail on malformed responses.
- Sorting tests for string values are case-insensitive to ensure consistent ordering.
- Price-based tests convert the price to a number before comparison to handle cases where it might be returned as a string.

3.4 API 03 - GET /products/{productId}

3.4.1 Overview

This endpoint retrieves a single product by its unique ID. It's designed to be robust, handling various ID formats and providing clear feedback for both valid and invalid requests. The tests cover schema validation, caching headers, and edge cases for the product ID.

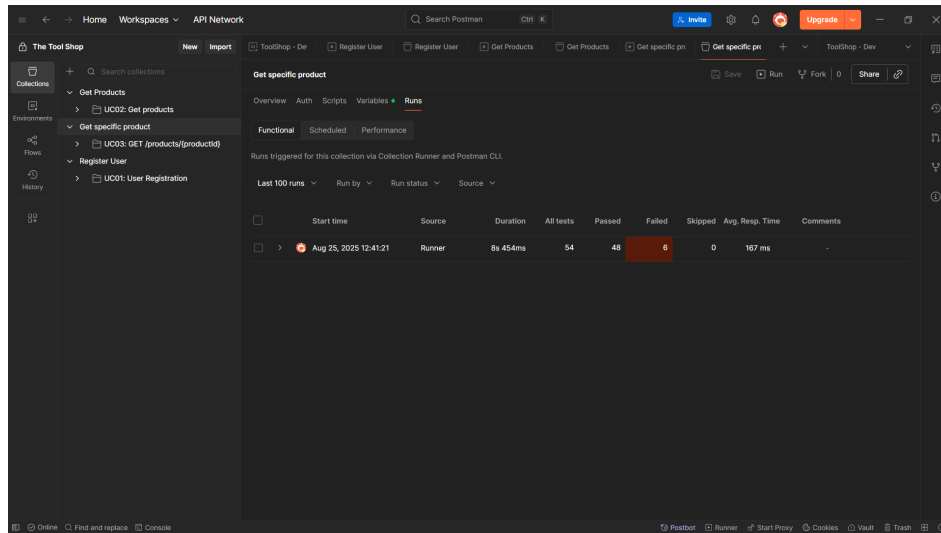


Figure 4: Get specific product collection in Postman

3.4.2 How to Test with Postman

1. Request Details

- **Method:** GET
- **URL:** `{{baseUrl}}/products/{{productId}}`
- **Headers:**
 - **Accept:** `application/json`
- **Path Variable:**
 - **productId:** The unique identifier for the product.

2. Test Scripts

- **Happy Path (TC301):** Checks for a 200 OK status and verifies that the response body is a JSON object with a valid `id`.
- **Schema Validation (TC302-TC303):** Ensures all expected fields are present and have the correct data types (e.g., `name` is a string, `price` is number-like).
- **Caching (TC305-TC307):** Validates the presence of `ETag` and `Last-Modified` headers and tests conditional GET requests using `If-None-Match` and `If-Modified-Since` to ensure a 304 Not Modified response when the resource is unchanged.
- **Negative Cases:**
 - **Invalid IDs (TC313-TC318):** Tests with non-existent, zero, negative, and non-integer IDs to ensure the API returns a 404 Not Found or 400 Bad Request.
 - **Security (TC320):** Includes a basic check for SQL injection-like patterns in the ID to ensure they are handled gracefully.

3.4.3 Example Test Cases

- **TC301:** Retrieve a product with a valid ID.
- **TC302:** Verify the presence of all top-level fields in the response.
- **TC306:** Test conditional GET with `If-None-Match` header.
- **TC313:** Request a product with a non-existent ID.
- **TC315:** Use a negative number for the product ID.
- **TC316:** Use a non-integer string for the product ID.

3.4.4 Expected Responses

- **200 OK:** The product is found, and the response contains its details.
- **304 Not Modified:** The product has not been modified since the last request (used with caching headers).
- **404 Not Found:** The requested product ID does not exist.
- **400 Bad Request:** The product ID is invalid (e.g., not an integer).

3.4.5 Additional Notes

- The collection uses variables for different types of IDs (`valid_id`, `non_exist_id`, `invalid_id_string`, etc.) to make tests easy to manage and run.
- Tests for caching headers are included to promote efficient API usage.
- A variety of invalid inputs are tested to ensure the API is resilient and secure.

4 AI supports

Throughout this project, ChatGPT was utilized as a productivity tool to accelerate several key testing activities. By providing targeted prompts, we were able to automate repetitive tasks, generate comprehensive test scenarios, and streamline documentation.

4.1 Test Case Generation

The initial step involved brainstorming and defining comprehensive test scenarios. ChatGPT was instrumental in this phase, helping to identify a wide range of test cases, including happy paths, negative scenarios, and edge cases that might otherwise be overlooked.

4.1.1 Example Prompt for Test Case Generation

Generate a list of test cases for the API endpoint `'GET /products/{productId}'`. Include tests for:

- Happy path (valid ID)
- Schema validation (field presence and types)
- Caching headers (ETag, If-None-Match)
- Negative cases (non-existent ID, zero, negative ID)
- Invalid input types (float, string)
- Security (SQL injection attempt)

4.2 Test Collection Generation

Once the test cases were defined and documented in a spreadsheet, we used that sheet as a source of truth to generate the Postman collections. By providing the structured test case data to ChatGPT, we could create the corresponding JSON for the entire collection in one step.

4.2.1 Example Prompt for Test Collection Generation

Based on the following test cases from my spreadsheet, generate a Postman collection as a JSON object for the API endpoint `'POST /users/register'`.

The base URL should be a variable `'{{baseUrl}}'`. For TC101, include a pre-request script to generate a random email. For all tests, include a script to assert the expected status code.

This two-step process—generating cases first, then collections—ensured our automated tests directly mapped to our documented requirements.

4.3 Summarize Defects and Actual Results

After running the tests, we used ChatGPT to process the results and generate clear, concise summaries for our documentation. This was particularly useful for filling out the “Actual Result” column in our test case spreadsheets and for creating detailed bug reports.

4.3.1 Example Prompt for Summarizing a Bug

Based on the following Postman test result for ‘TC104’, write a bug report summary and an "Actual Result" description.

```
**Request:**
- Method: POST
- URL: ‘{{baseUrl}}/users/register‘
- Body: ‘{ "email": "invalid-email.com", ... }‘

**Test Script:**
‘pm.test("Status code is 400/422", () => {
  pm.expect(pm.response.code).to.be.oneOf([400, 422]);
});‘

**Result:**
- Status: 500 Internal Server Error
- Test Result: FAILED - ‘AssertionError:
  expected 500 to be one of [ 400, 422 ]‘

**Summary to Generate:**
1. A formal bug report summary.
2. A concise "Actual Result" description.
```

4.3.2 Generated Output Example

- **Bug Report Summary:** The API returns a 500 Internal Server Error when a user attempts to register with an invalid email format (e.g., missing the “@” symbol). The expected behavior is a 400 Bad Request or 422 Unprocessable Entity with a clear validation error message. The current implementation fails to handle the input validation gracefully, leading to an unhandled exception.
- **Actual Result:** The test failed. The API responded with a 500 Internal Server Error instead of the expected 400 or 422 client error.

5 Test results

5.1 API 01 - POST /users/register

Execution for this API revealed 12 defects, primarily related to input validation and incorrect server responses. Key issues include failure to reject invalid data formats

(email, phone, postcode), improper handling of edge cases like future dates and client-provided IDs, and returning incorrect status codes for both valid and invalid requests.

The following test cases passed successfully: - **TC102:** Negative - Duplicate Email - **TC103:** Negative - Missing Required Field

Defects Found:

Test Case ID	Description	Status	Notes
TC101	Happy Path - Successful Registration	FAILED	Defect B1000: Response body <code>unique_id</code> is not a string.
TC104	Negative - Invalid Email Format (no @)	FAILED	Defect B1001: Returned 500 Internal Server Error instead of 400/422.
TC105	Negative - Invalid Email Format (no domain)	FAILED	Defect B1002: Returned 201 Created instead of 400/422.
TC110	Negative - Invalid Date Value (Future Date)	FAILED	Defect B1003: Error message did not clearly indicate "dob not allowed".
TC118	Edge Case - id Field in Request	FAILED	Defect B1004: The server did not ignore the client-provided id.
TC122	Validation - Invalid Phone Number (letters)	FAILED	Defect B1005: Returned 201 Created instead of 400/422.
TC123	Validation - Postcode with special characters	FAILED	Defect B1006: Returned 201 Created; should have rejected special characters.
TC124	Validation - Unicode Characters in Name	FAILED	Defect B1007: Returned 422; Unicode characters were not accepted.
TC125	Validation - Case-Insensitive Duplicate Email	FAILED	Defect B1008: Error did not clearly indicate a duplicate email.

Test Case ID	Description	Status	Notes
TC126	Security - Unauthorized Access Attempt	FAILED	Defect B1009: Returned 201 Created instead of 401 Unauthorized.
TC130	Negative - Country field with numbers	FAILED	Defect B1010: Returned 422, but the error message was misleading.
TC131	Validation - Postcode with spaces	FAILED	Defect B1011: Returned 422; <code>postal_code</code> was missing from the response.

5.1.1 Evidence

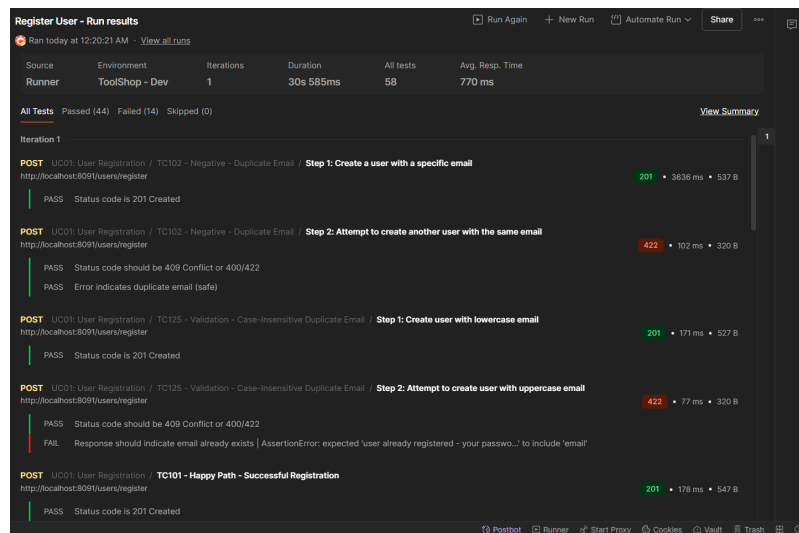


Figure 5: API 01 Evidence 1

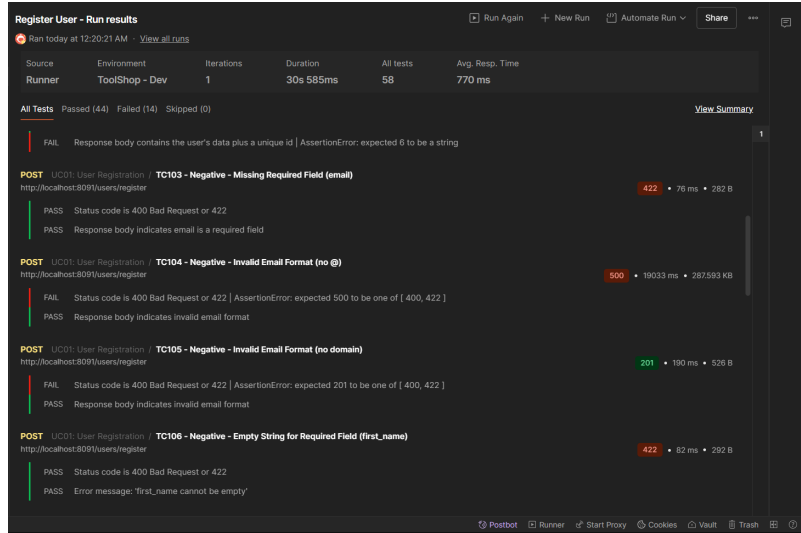


Figure 6: API 01 Evidence 2

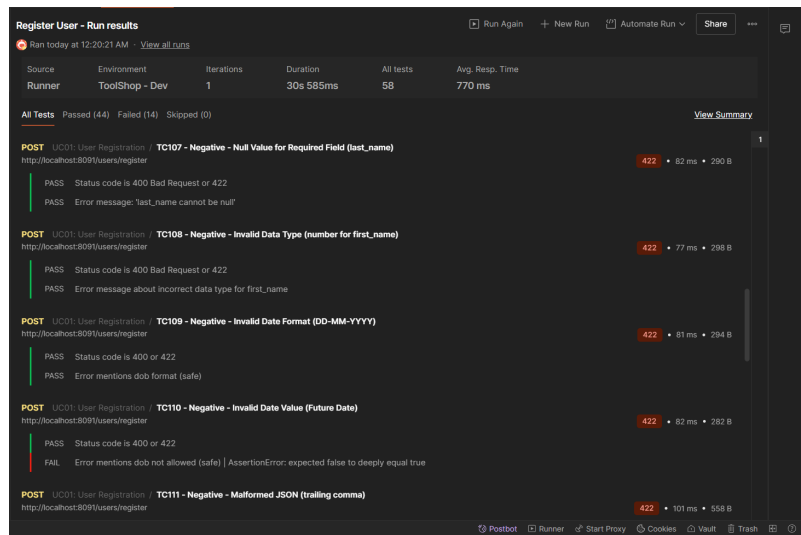


Figure 7: API 01 Evidence 3

Register User - Run results						Run Again	+ New Run	Automate Run	Share	...	
Run today at 12:20:21 AM - View all runs											
Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time						
Runner	ToolShop - Dev	1	30s 585ms	58	770 ms						
All Tests Passed (44) Failed (14) Skipped (0)						View Summary					
POST UC01: User Registration / TC111 - Negative - Malformed JSON (trailing comma)						1					
http://localhost:8091/users/register						422 • 101 ms • 558 B					
PASS Status code is 400 or 422											
POST UC01: User Registration / TC112 - Negative - Empty Request Body						422 • 73 ms • 558 B					
http://localhost:8091/users/register											
PASS Status code is 400 Bad Request or 422											
PASS Error message indicates required fields are missing											
POST UC01: User Registration / TC113 - Negative - No Request Body						422 • 71 ms • 558 B					
http://localhost:8091/users/register											
PASS Status code is 400 or 422											
POST UC01: User Registration / TC114 - Security - SQL Injection Attempt						422 • 80 ms • 298 B					
http://localhost:8091/users/register											
PASS Status is 400/422 or 201 (if sanitized)											
POST UC01: User Registration / TC115 - Security - Cross-Site Scripting (XSS) Attempt						201 • 164 ms • 555 B					
http://localhost:8091/users/register											
PASS Status code is 201 Created											
PASS Response body shows the address as an escaped/raw string, not active content											
						Postbot Runner Start Proxy Cookies Vault Trash					

Figure 8: API 01 Evidence 4

Register User - Run results						Run Again	+ New Run	Automate Run	Share	...	
Run today at 12:20:21 AM - View all runs											
Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time						
Runner	ToolShop - Dev	1	30s 585ms	58	770 ms						
All Tests Passed (44) Failed (14) Skipped (0)						View Summary					
POST UC01: User Registration / TC116 - Boundary - Very Long String Input						1					
http://localhost:8091/users/register						422 • 81 ms • 319 B					
PASS Status code is 400 Bad Request or 422											
PASS Error message: 'first_name exceeds maximum length'											
POST UC01: User Registration / TC117 - Edge Case - Extra Unexpected Field						201 • 190 ms • 539 B					
http://localhost:8091/users/register											
PASS Status code is 201 Created											
PASS The server should ignore the unexpected field											
POST UC01: User Registration / TC118 - Edge Case - Id Field in Request						201 • 202 ms • 540 B					
http://localhost:8091/users/register											
PASS Status code is 201 Created											
FAIL The server should ignore the client-provided id AssertionError: expected 10 to be a string											
GET UC01: User Registration / TC119 - API Contract - Wrong HTTP Method (GET)						401 • 80 ms • 257 B					
http://localhost:8091/users/register											
PASS Status code is 401 Unauthorized											
PUT UC01: User Registration / TC120 - API Contract - Wrong HTTP Method (PUT)						401 • 76 ms • 257 B					
http://localhost:8091/users/register											
PASS Status code is 401 Unauthorized											
						Postbot Runner Start Proxy Cookies Vault Trash					

Figure 9: API 01 Evidence 5

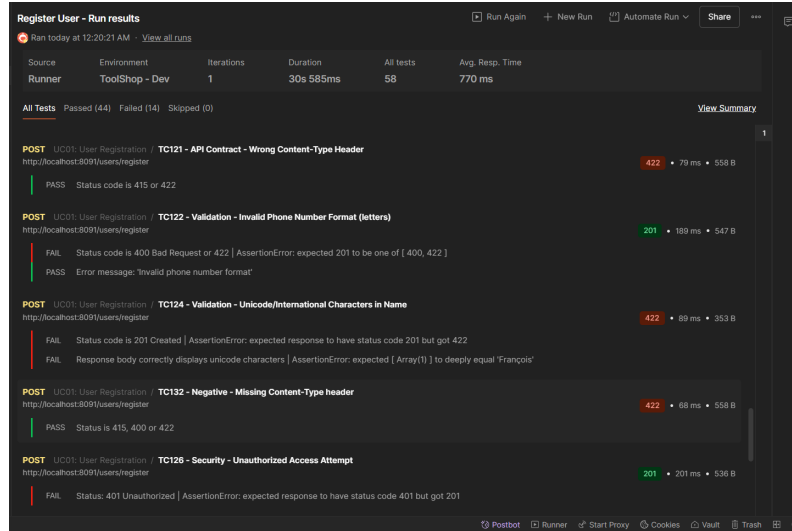


Figure 10: API 01 Evidence 6

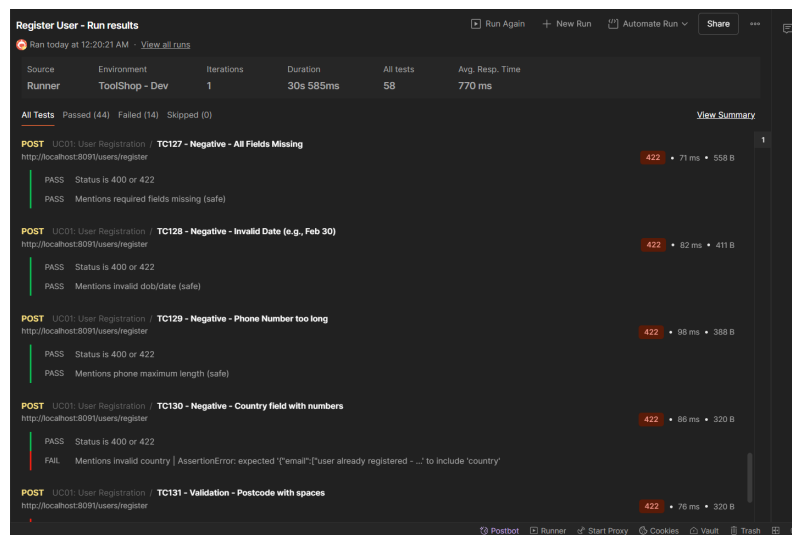


Figure 11: API 01 Evidence 7

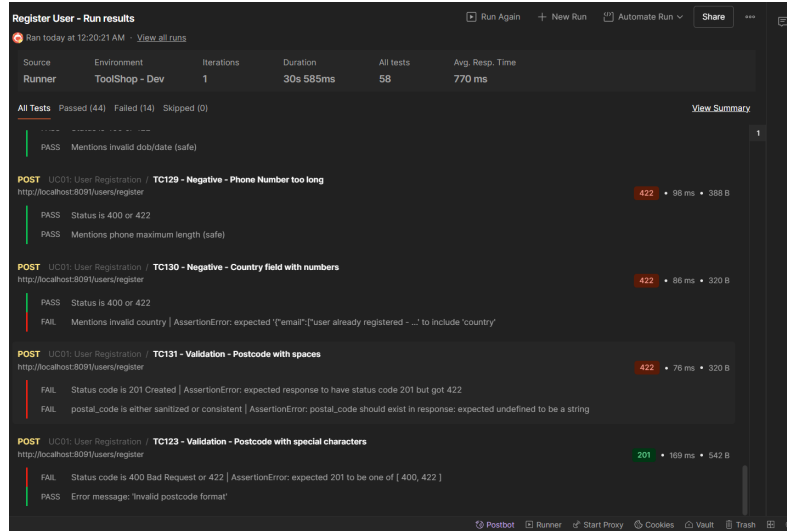


Figure 12: API 01 Evidence 8

5.2 API 02 - GET /products

Testing for this endpoint passed for all primary filter and sort functionalities (**TC201-TC208**). However, 9 defects were discovered in how the API handles invalid parameters. The API often returned a 200 OK with an empty or incorrect list instead of a 400 Bad Request or 422 Unprocessable Entity, indicating a lack of robust input validation for query parameters.

Defects Found:

Test Case ID	Description	Status	Notes
TC214	Negative - Invalid Brand ID Format (string)	FAILED	Defect B1012: Returned 200 OK instead of a validation error.
TC215	Negative - Invalid between Format (Missing Value)	FAILED	Defect B1013: Returned 200 OK; ignored malformed parameter.
TC216	Negative - Invalid between Format (Wrong Column)	FAILED	Defect B1014: Returned 200 OK; accepted a non-price column.
TC218	Negative - Invalid sort Column	FAILED	Defect B1015: Returned 200 OK; accepted an invalid sort column.
TC219	Negative - Invalid sort Direction	FAILED	Defect B1016: Returned 200 OK; accepted an invalid sort direction.

Test Case ID	Description	Status	Notes
TC220	Negative - Invalid <code>is_rental</code> Value	FAILED	Defect B1017: Returned 200 OK; accepted 'yes' instead of 0/1.
TC221	API Contract - Wrong Method (POST)	FAILED	Defect B1018: Returned 422 instead of 405 Method Not Allowed.
TC225	Pagination - Invalid Page Number Format	FAILED	Defect B1019: Returned 200 OK; accepted a non-numeric page value.
TC226	Security - SQL Injection in <code>sort</code>	FAILED	Defect B1020: Returned 200 OK; did not sanitize the SQL-like payload.

5.2.1 Evidence

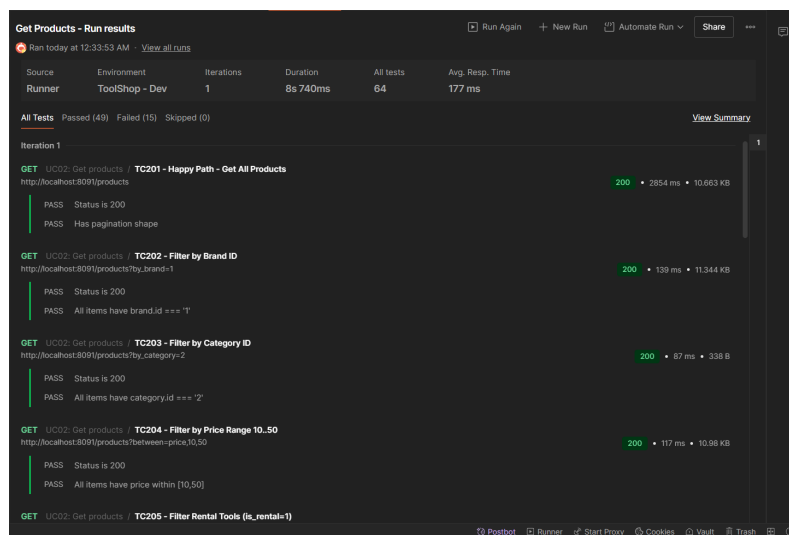


Figure 13: API 02 Evidence 1

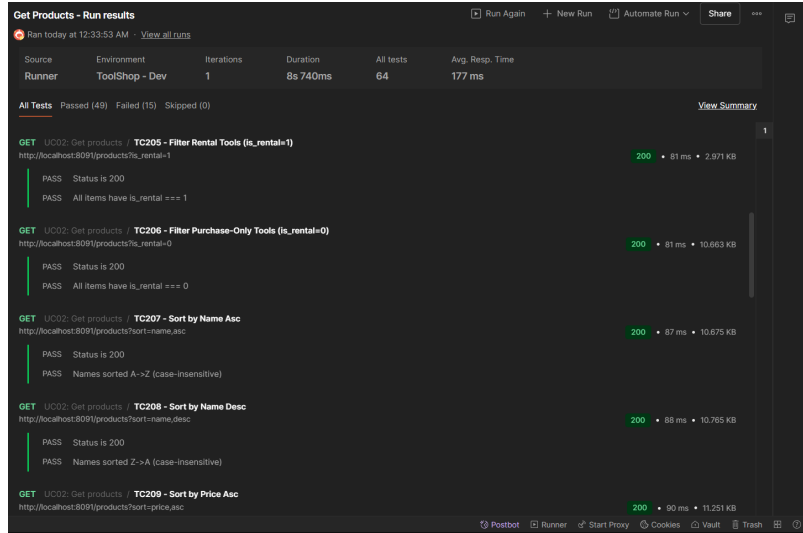


Figure 14: API 02 Evidence 2

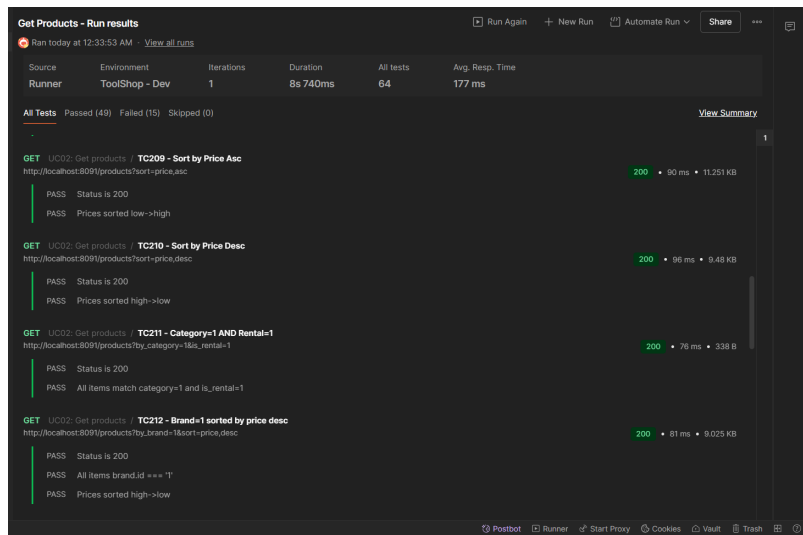


Figure 15: API 02 Evidence 3

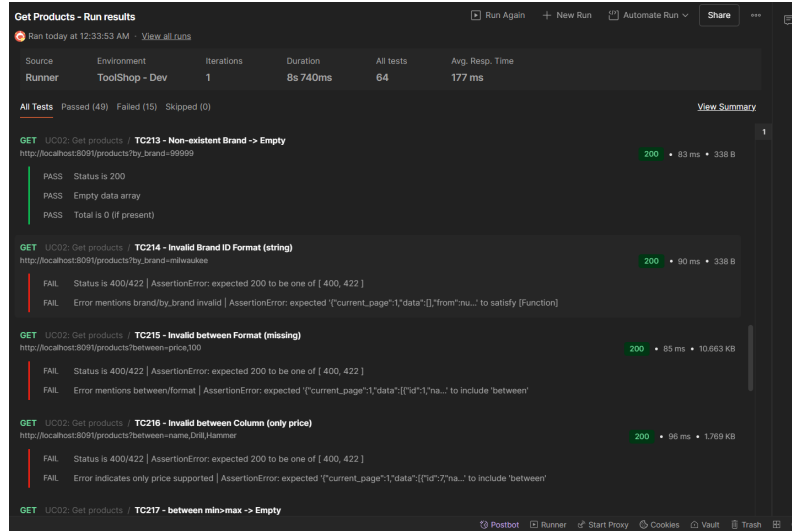


Figure 16: API 02 Evidence 4

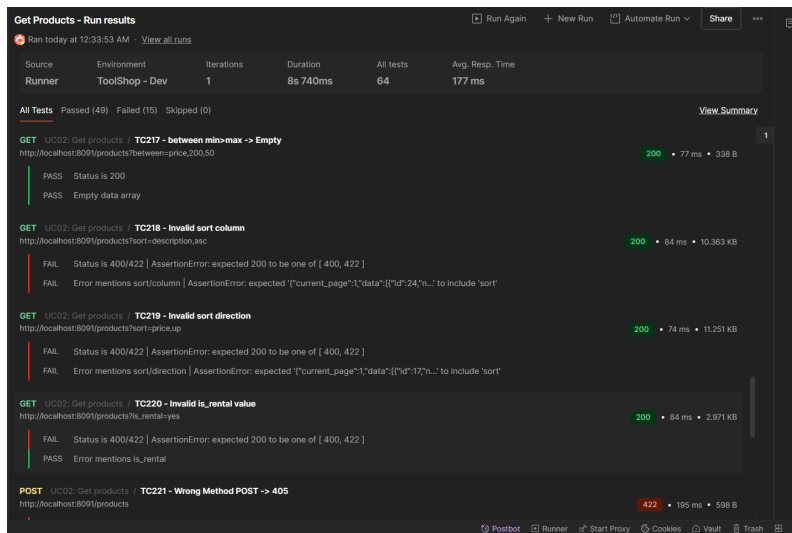


Figure 17: API 02 Evidence 5

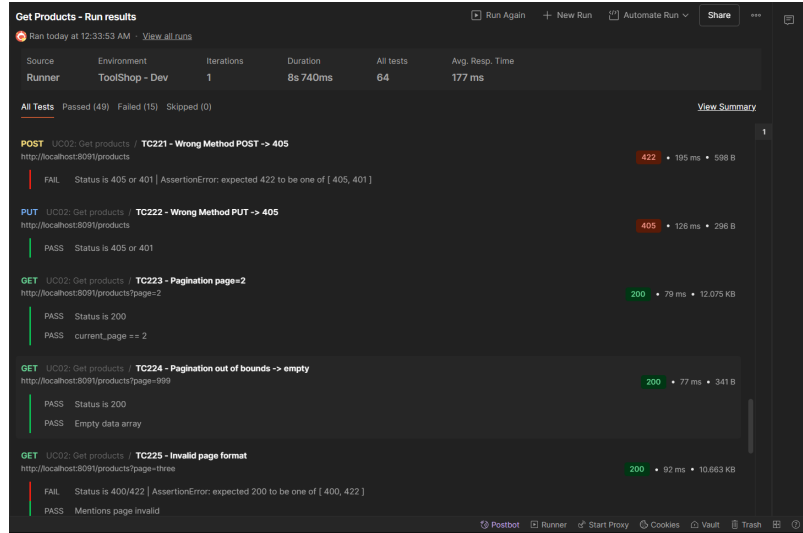


Figure 18: API 02 Evidence 6

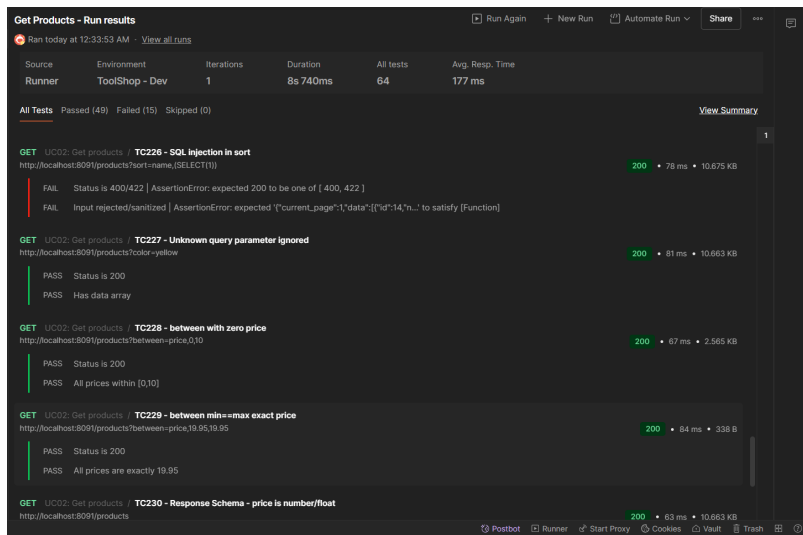


Figure 19: API 02 Evidence 7

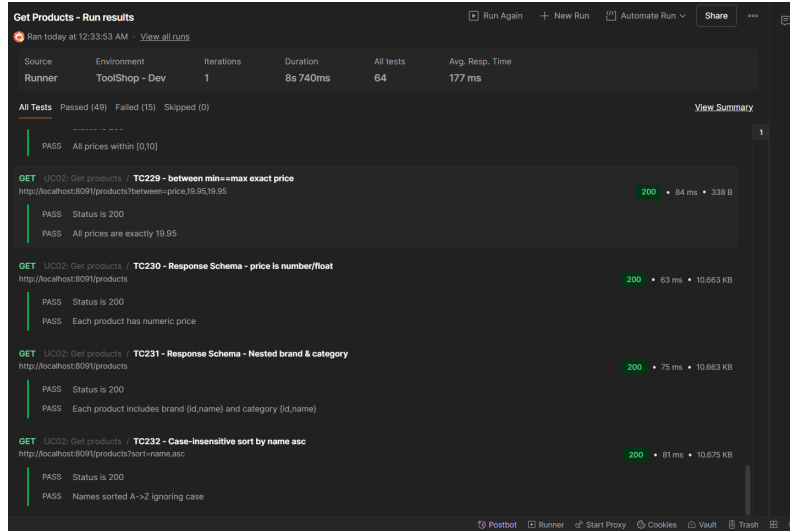


Figure 20: API 02 Evidence 8

5.3 API 03 - GET /products/{productId}

The core functionality of retrieving a product by a valid ID was successful. However, 6 defects were found related to protocol handling and security. The API failed to implement caching correctly, did not handle URL variations like trailing slashes gracefully, and failed to reject requests using incorrect HTTP methods.

The following test cases passed successfully: - **TC301-TC304:** Happy Path and Schema Validation - **TC306-TC310:** Conditional GETs and Header negotiation - **TC312-TC313:** Invalid/Non-Existent ID - **TC315-TC320:** Invalid ID formats and Security

Defects Found:

Test Case ID	Description	Status	Notes
TC305	Caching – ETag present	FAILED	Defect B1021: ETag header was missing from the response.
TC311	Trailing slash	FAILED	Defect B1022: Returned 404 Not Found; did not redirect.
TC314	Unknown query parameter is ignored	FAILED	Defect B1023: Returned 404 Not Found; did not ignore the parameter.
TC322	ID – URL encoded valid	FAILED	Defect B1024: Returned 404 Not Found; did not accept a valid URL-encoded ID.

Test Case ID	Description	Status	Notes
TC329	Wrong method – PUT	FAILED	Defect B1025: Returned 200 OK instead of 405/401.
TC330	Wrong method – DELETE	FAILED	Defect B1026: Returned 409 Conflict instead of 405/401.

5.3.1 Evidence

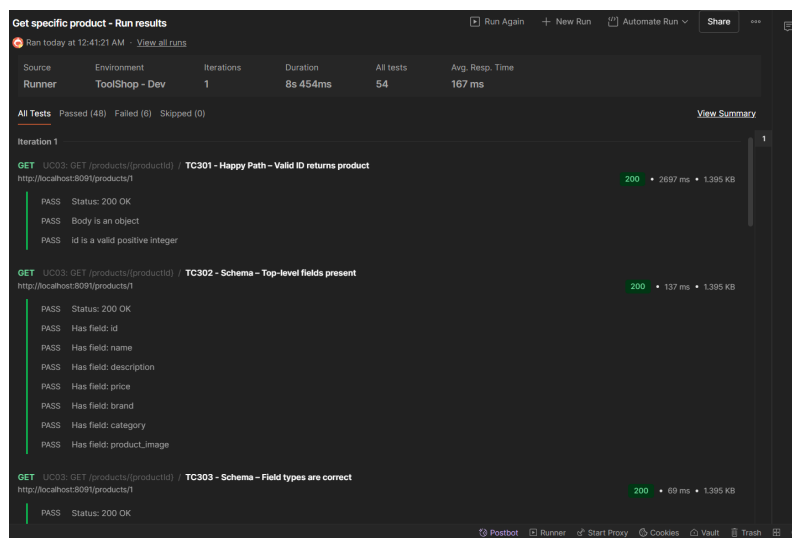


Figure 21: API 03 Evidence 1

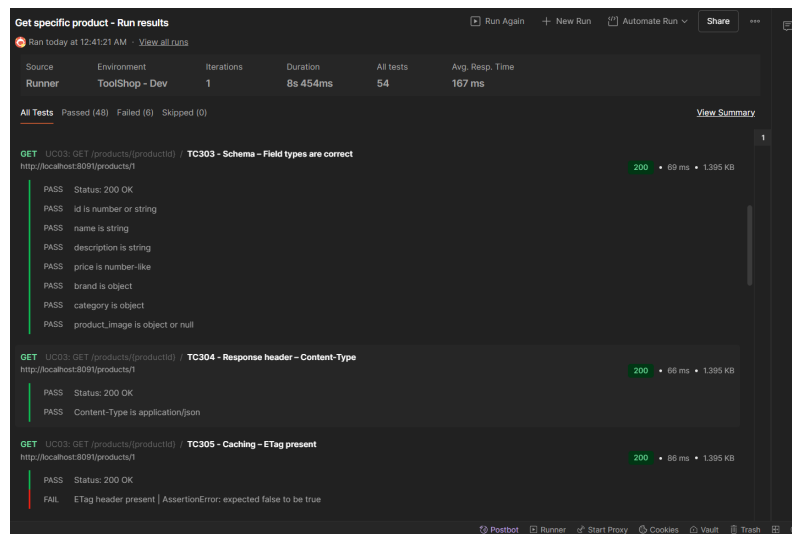


Figure 22: API 03 Evidence 2

Get specific product - Run results						Run Again	+ New Run	Automate Run	Share	...
Ran today at 12:41:21 AM - View all runs										
Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time					
Runner	ToolShop - Dev	1	8s 454ms	54	167 ms					
All Tests Passed (48) Failed (6) Skipped (0)						View Summary				
GET UC03: GET /products/{productId} / TC306 - Conditional GET - If-None-Match						1				
http://localhost:8091/products/1						200 • 72 ms • 1.395 KB				
PASS Status: 200 OK (initial fetch)										
PASS Server does not provide ETag (skip)										
GET UC03: GET /products/{productId} / TC307 - Conditional GET - If-Modified-Since						200 • 76 ms • 1.395 KB				
http://localhost:8091/products/1										
PASS Status: 200 OK (initial fetch)										
PASS Server does not provide Last-Modified (skip)										
GET UC03: GET /products/{productId} / TC308 - Accept header - unsupported (XML)						200 • 70 ms • 1.395 KB				
http://localhost:8091/products/1										
PASS Either 406, or 200 JSON fallback										
PASS Fallback Content-Type is application/json										
GET UC03: GET /products/{productId} / TC309 - Accept header - wildcard */*						200 • 94 ms • 1.395 KB				
http://localhost:8091/products/1										
PASS Status: 200 OK										
GET UC03: GET /products/{productId} / TC310 - Case sensitivity - /PRODUCTS path						404 • 128 ms • 260 B				
http://localhost:8091										
PASS Status: 404 Not Found										

Figure 23: API 03 Evidence 3

Get specific product - Run results						Run Again	+ New Run	Automate Run	Share	...
Ran today at 12:41:21 AM - View all runs										
Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time					
Runner	ToolShop - Dev	1	8s 454ms	54	167 ms					
All Tests Passed (48) Failed (6) Skipped (0)						View Summary				
GET UC03: GET /products/{productId} / TC311 - Trailing slash						1				
http://localhost:8091						404 • 71 ms • 260 B				
FAIL Accept 200 or redirect 301/302 AssertionError: expected [200, 301, 302] to include 404										
GET UC03: GET /products/{productId} / TC312 - Duplicate slashes in path						404 • 87 ms • 260 B				
http://localhost:8091										
PASS 404 or 400 for malformed path										
GET UC03: GET /products/{productId} / TC313 - Extra path segment						404 • 70 ms • 260 B				
http://localhost:8091										
PASS Status: 404 Not Found										
GET UC03: GET /products/{productId} / TC314 - Unknown query parameter is ignored						404 • 75 ms • 260 B				
http://localhost:8091										
FAIL Status: 200 OK AssertionError: expected response to have status code 200 but got 404										
GET UC03: GET /products/{productId} / TC315 - ID - lowercase accepted						200 • 78 ms • 1.395 KB				
http://localhost:8091/products/1										
PASS Status: 200 OK										
GET UC03: GET /products/{productId} / TC316 - ID - negative number						404 • 107 ms • 266 B				
http://localhost:8091/products/-1										

Figure 24: API 03 Evidence 4

Get specific product - Run results					
Run today at 12:41:21 AM · View all runs					
Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	ToolShop - Dev	1	8s 454ms	54	167 ms
All Tests Passed (48) Failed (8) Skipped (0) View Summary					
1					
GET UC03: GET /products/{productId} / TC316 - ID - negative number					
http://localhost:8091/products/-1					
PASS 400 or 404 for invalid id					
GET UC03: GET /products/{productId} / TC317 - ID - zero value					
http://localhost:8091/products/0					
PASS 400 or 404 for invalid id					
GET UC03: GET /products/{productId} / TC318 - ID - non-numeric string					
http://localhost:8091/products/abc					
PASS 400 or 404 for invalid id					
GET UC03: GET /products/{productId} / TC319 - ID - floating point number					
http://localhost:8091/products/1.5					
PASS 400 or 404 for invalid id					
GET UC03: GET /products/{productId} / TC320 - ID - special characters					
http://localhost:8091/products/@					
PASS 400 or 404 for invalid id					
GET UC03: GET /products/{productId} / TC321 - ID - quoted in path					
http://localhost:8091					
PASS 400 or 404 for invalid id					

Figure 25: API 03 Evidence 5

Get specific product - Run results					
Run today at 12:41:21 AM · View all runs					
Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	ToolShop - Dev	1	8s 454ms	54	167 ms
All Tests Passed (48) Failed (6) Skipped (0) View Summary					
1					
GET UC03: GET /products/{productId} / TC321 - ID - quoted in path					
http://localhost:8091					
PASS Status: 404 Not Found					
GET UC03: GET /products/{productId} / TC322 - ID - URL encoded valid					
http://localhost:8091					
FAIL Status: 200 OK AssertionError: expected response to have status code 200 but got 404					
GET UC03: GET /products/{productId} / TC323 - ID - URL encoded invalid (SQL-like)					
http://localhost:8091					
PASS 400 or 404 for invalid id					
GET UC03: GET /products/{productId} / TC324 - Security - path traversal attempt					
http://localhost:8091					
PASS 404 or 400 for path traversal					
GET UC03: GET /products/{productId} / TC325 - Security - null byte injection					
http://localhost:8091					
PASS 400 or 404 for invalid id					
GET UC03: GET /products/{productId} / TC326 - Security - semicolon matrix param					
http://localhost:8091					
PASS 404 or 400 for invalid id					

Figure 26: API 03 Evidence 6

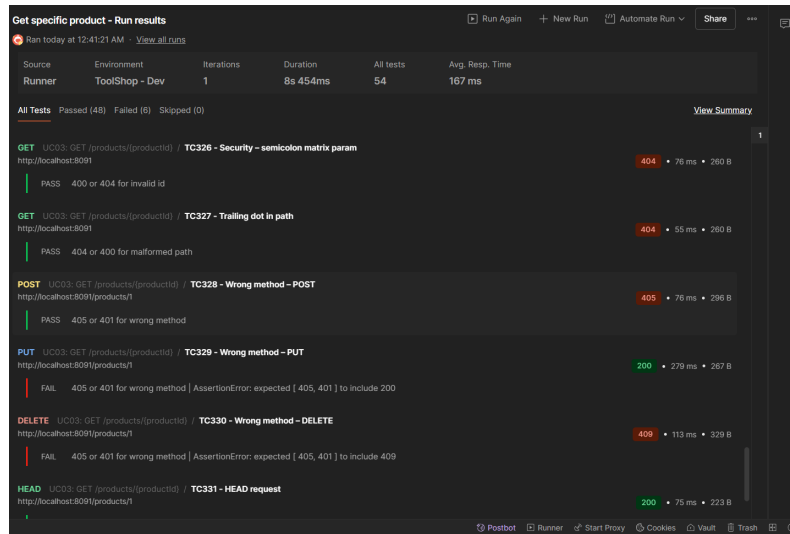


Figure 27: API 03 Evidence 7

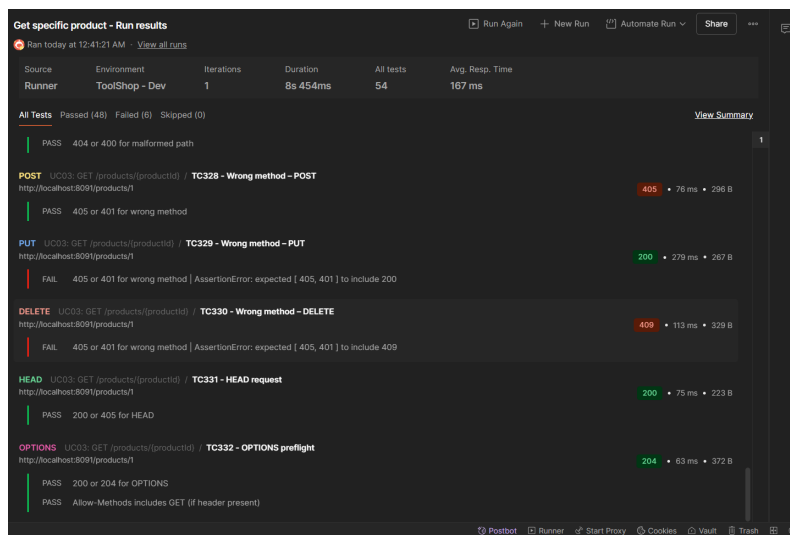


Figure 28: API 03 Evidence 8

6 API Testing CI Integration

6.1 Overview

This section documents the integration of automated API testing into the CI/CD pipeline using Newman (Postman CLI) and GitHub Actions workflows.

6.2 Implementation

6.2.1 Technology Stack

- **Newman:** Command-line runner for Postman collections
- **GitHub Actions:** CI/CD automation platform
- **Docker:** Containerized test environment
- **HTML Extra Reporter:** Enhanced test reporting

6.2.2 Workflow Architecture

Four GitHub Actions workflows were implemented:

1. **api-tests.yml**: Dedicated API testing workflow
2. **complete-tests.yml**: Combined API and E2E testing
3. **sprint5-bugs-api-tests.yml**: Bug discovery testing for sprint5-with-bugs
4. **run-tests.yml**: Enhanced original workflow with API testing integration

6.2.3 Test Coverage

- **UC01**: User Registration API (POST /users/register)
- **UC02**: Get Products API (GET /products)
- **UC03**: Get Specific Product API (GET /products/{id})

Each test collection includes validation for status codes, response structure, data integrity, and error handling.

6.3 Technical Configuration

6.3.1 Newman Execution

```
newman run collection.json
-e environment.json
--reporters cli,json,htmlextra
--timeout-request 30000
--delay-request 1000
```

6.3.2 Environment Setup

- **Local Development**: http://localhost:8091
- **Sprint5-With-Bugs**: http://localhost:8093 (modified ports)
- **CI Environment**: GitHub Actions Ubuntu runners with Docker

6.3.3 Reporting

- **HTML Reports**: Comprehensive test execution details
- **JSON Results**: Machine-readable outcomes for analysis
- **Artifacts**: 30-day retention in GitHub Actions
- **Cross-Platform Scripts**: Bash and PowerShell support for local development

6.4 Results

The integration successfully provides automated API testing capabilities with comprehensive reporting, cross-platform support, and seamless CI/CD integration. The implementation supports both standard development workflows and specialized testing scenarios for educational purposes.

6.4.1 Evidence

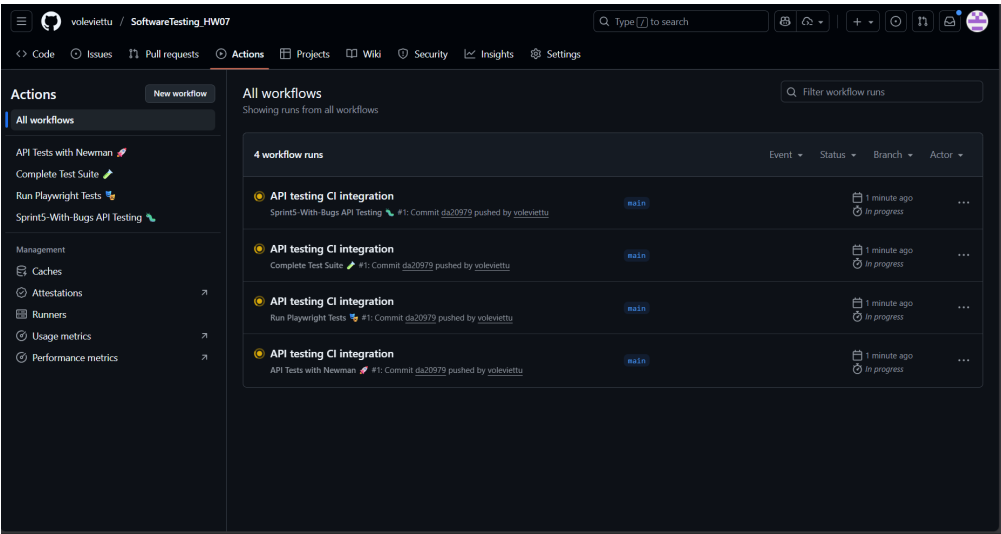


Figure 29: CI Workflow Evidence 1

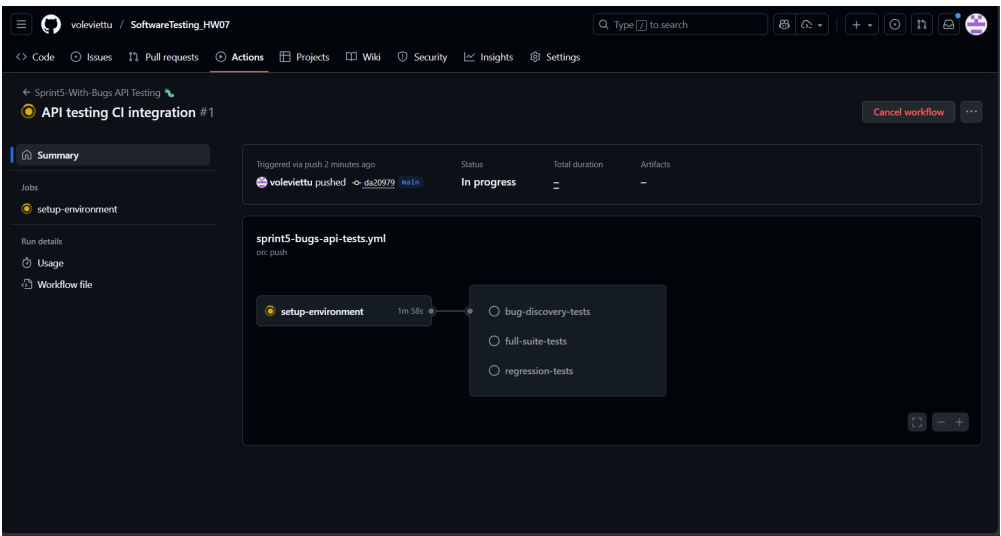


Figure 30: CI Workflow Evidence 2

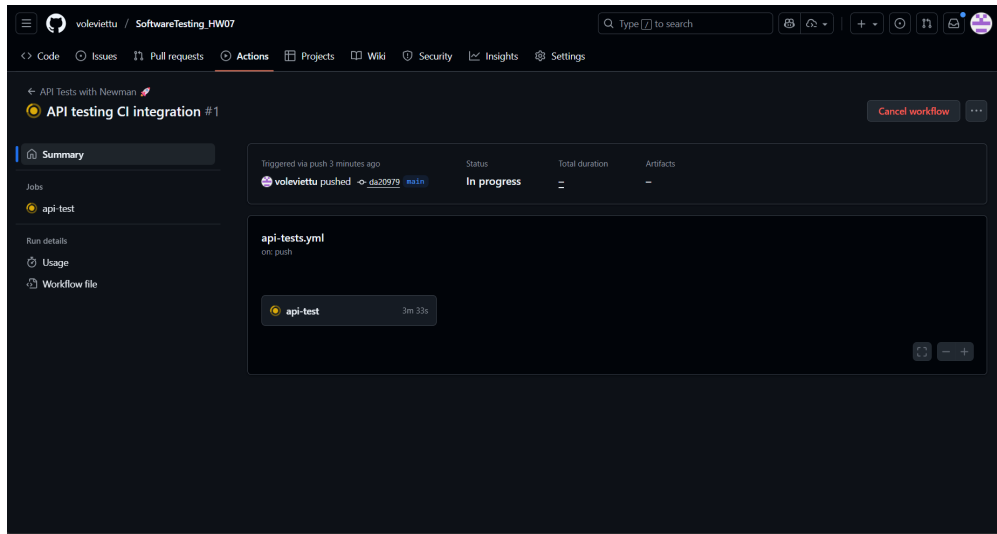


Figure 31: CI Workflow Evidence 3

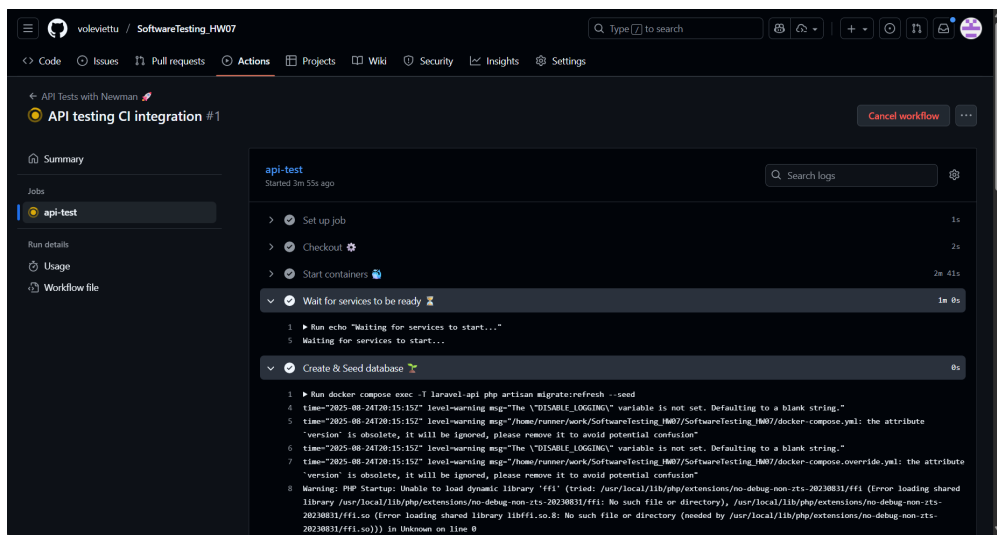


Figure 32: CI Workflow Evidence 4

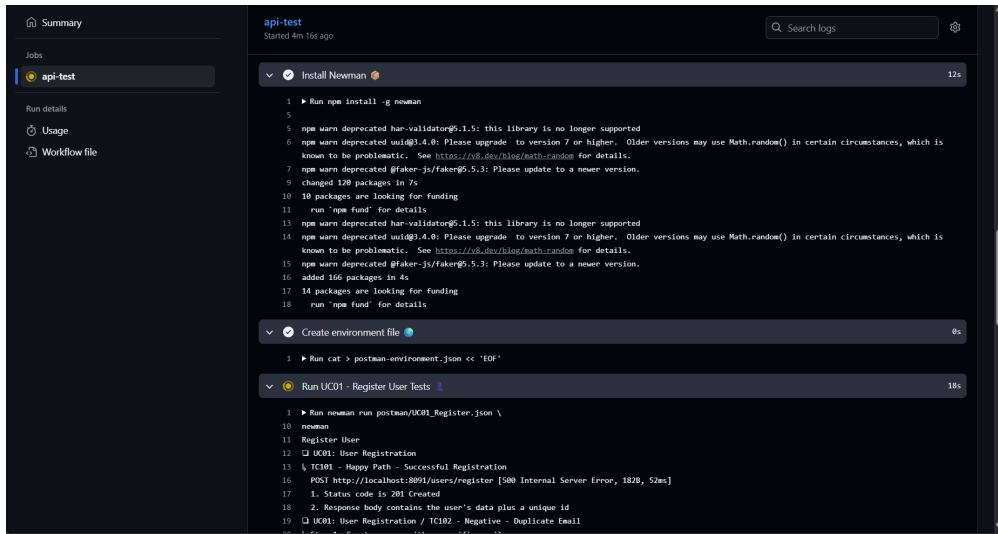


Figure 33: CI Workflow Evidence 5

7 Self-assessment

Score Range	Criteria for each API
8 – 10	Comprehensive step-by-step explain with annotated screenshots ≥ 30 well-classified test cases Full CI-workflow evidence Detailed bug reports with reproduction steps AI supports are well documented
5 – 7	Report is clear but not comprehensive, few screenshots ≤ 20 well-classified test cases Little CI-workflow evidence Detailed bug reports with reproduction steps
Below 5	Minimal AI proofing Incomplete or unclear report ≤ 5 test cases Few or no bug reports No CI workflow evidence No AI proofing

Outcomes (Brief description about what you Criteria get/trouble from each requirement)		Self-Assessed Percent Grade	
1 API		30%	
1			
1.1	Report is comprehensive, step-by-step with annotated screenshots	15	15
1.2	More than 30 well-classified test cases are provided	5	5
Test Cases (30)			

Criteria	Outcomes (Brief description about what you get/trouble from each requirement)	Percent	Self-Assessed Grade
1.3 Screen- shots on Test- ing Tools	Clear screenshots with evidence of Postman/Newman tests	5	5
1.4 Bugs	Detailed bug reports with reproduction steps	5	5
2 API 2		30%	
2.1 Re- port	Report is comprehensive, step-by-step with annotated screenshots	15	15
2.2 Test Cases (30)	More than 30 well-classified test cases are provided	5	5
2.3 Screen- shots on Test- ing Tools	Clear screenshots with evidence of Postman/Newman tests	5	5
2.4 Bugs	Detailed bug reports with reproduction steps	5	5
3 API 3		30%	
3.1 Re- port	Report is comprehensive, step-by-step with annotated screenshots	15	15
3.2 Test Cases (30)	More than 30 well-classified test cases are provided	5	5
3.3 Screen- shots on Test- ing Tools	Clear screenshots with evidence of Postman/Newman tests	5	5

Criteria	Outcomes (Brief description about what you get/trouble from each requirement)	Percent	Self-Assessed Grade
3.4 Bugs	Detailed bug reports with reproduction steps	5	5
4 Well formatted	Report is well structured and formatted consistently	10%	10
Total		100%	100%